

Symfony

Notes techniques Symfony. Une section par sujet documenté.

Cache et environnements multi-tenant (Symfony Runtime / Doctrine / Twig)

Contexte

Paramétrage effectué pour le projet **Synoptic** (application B2B de prise de commande, Symfony 7.2).

Synoptic utilise le **nom de l'environnement Symfony comme identifiant de tenant** : `synoptic`, `preprod`, et potentiellement d'autres clients à venir, en plus des environnements standards `dev` / `test`. Chaque environnement porte sa config métier (base de données, couleurs, mailer...) via un fichier `.env.<env>` dédié. Ce choix architectural a des effets de bord sur le cache, car plusieurs mécanismes internes de Symfony sont câblés sur le nom **littéral** `prod`, pas sur la notion générale "environnement de production".

Problème 1 — Le cache Doctrine ne s'activait que sous `prod`

Dans `config/packages/doctrine.yaml`, les optimisations ORM (`auto_generate_proxy_classes: false`, `query_cache_driver`, `result_cache_driver` via pools) étaient déclarées uniquement sous un bloc `when@prod:`. Les blocs `when@xxx:` de Symfony font un **match littéral** sur `kernel.environment` : comme l'environnement s'appelle `synoptic` et pas `prod`, ce bloc ne s'appliquait jamais en environnement synoptic — Doctrine régénérait les proxies à la volée et n'avait ni query cache ni result cache.

Piège rencontré en corrigeant : Symfony ne supporte **pas** de syntaxe

`when@prod||when@synoptic:` pour combiner plusieurs environnements sur une même clé (vérifié dans le loader YAML de `symfony/dependency-injection`, qui ne reconnaît que la clé exacte `when@<env>`). Il faut donc dupliquer le bloc pour chaque environnement prod-like :

```
when@prod:
    doctrine:
```

```

orm:
    auto_generate_proxy_classes: false
    proxy_dir: '%kernel.build_dir%/doctrine/orm/Proxies'
    query_cache_driver:
        type: pool
        pool: doctrine.system_cache_pool
    result_cache_driver:
        type: pool
        pool: doctrine.result_cache_pool

framework:
    cache:
        pools:
            doctrine.result_cache_pool: { adapter: cache.app }
            doctrine.system_cache_pool: { adapter: cache.system }

when@synoptic:
    doctrine: *même bloc*
    framework: *même bloc*

when@preprod:
    doctrine: *même bloc*
    framework: *même bloc*
```

Vérification après coup : `php bin/console debug:config doctrine orm --env=synoptic` doit afficher `auto_generate_proxy_classes: false` et des `query_cache_driver` / `result_cache_driver` de type `pool`.

Problème 2 — `APP_DEBUG` non explicite → debug actif par défaut

Aucun fichier `.env*` du projet ne définissait `APP_DEBUG`. Or Symfony Runtime calcule sa valeur par défaut ainsi :

```
$debug = !\in_array($_SERVER['APP_ENV'], $prodEnvs, true);
```

avec `$prodEnvs = ['prod']` par défaut (pas de `extra.runtime.prod_envs` dans `composer.json`). Comme `synoptic` $\neq$ `prod`, ce test vaut `true` : **`APP_DEBUG=1` par défaut sur l'environnement synoptic, y compris en production réelle.**

Symptôme observé : modifier un template Twig est pris en compte immédiatement, sans vider le cache. Ce n'est pas anormal en soi — c'est `twig.auto_reload` (qui vaut par défaut `%kernel.debug%`) qui recompile automatiquement un template dès que son fichier source a été modifié — mais ce comportement n'est voulu qu'en développement, pas sur le vrai serveur.

Piège de la correction "évidente" : ajouter `synoptic / preprod` à `extra.runtime.prod_envs` dans `composer.json` aurait semblé logique, mais ce fichier est **versionné et partagé** entre le poste de dev et la prod. Résultat : le debug aurait aussi disparu localement dès qu'on travaille en `APP_ENV=synoptic` (cas courant ici, puisque l'environnement sert aussi de sélecteur de tenant en local). Cette solution a donc été écartée.

Solution retenue — séparer "tenant" et "type de machine" via Apache

La distinction "suis-je sur le vrai serveur du tenant, ou sur un poste de dev qui cible ce tenant" est portée par le `Host HTTP réel`, pas par le nom d'environnement. On la matérialise dans le `.htaccess` du serveur de production (fichier **non versionné**, géré à la main par serveur) :

```
SetEnvIf Host ^synoptic-v2.amshop.fr APP_ENV=synoptic
SetEnvIf Host ^synoptic-v2.amshop.fr APP_DEBUG=0
```

Pourquoi ça marche de façon fiable : `SetEnvIf` positionne la variable dans `$_SERVER` **avant** que Dotenv ne s'exécute. La méthode `populate()` de Symfony Dotenv copie d'abord `$_SERVER[name]` vers `$_ENV[name]`, puis refuse d'écraser une valeur qui n'a pas été positionnée par Dotenv lui-même. Donc toute valeur `APP_ENV=...` / `APP_DEBUG=...` déclarée dans un fichier `.env.*` du dépôt ne peut jamais écraser ce que Apache a déjà posé pour ce `Host`.

Sur le poste de dev, le serveur local (`symfony server:start`, lancé via `Host: 127.0.0.1:...`) ne matche jamais cette règle `Host` de prod — le debug local reste donc actif par défaut, sans configuration supplémentaire, quel que soit le tenant ciblé.

À dupliquer pour chaque nouveau tenant réel (`preprod`, etc.) avec le vrai domaine du vhost correspondant.

Ligne parasite trouvée dans `.env.synoptic`

Le fichier `.env.synoptic` (versionné, partagé entre dev et tous les serveurs) contenait une ligne `APP_ENV=prod` en tête de fichier — probablement un reliquat de copier-coller. Grâce au mécanisme de protection décrit ci-dessus, cette ligne était **inerte** tant qu'un `Host` réel positionnait déjà `APP_ENV` via Apache. Elle a quand même été commentée par prudence : elle constituait un piège pour tout futur appel qui n'aurait pas cette protection (voir piège CLI ci-dessous), et elle était trompeuse à la lecture.

Piège CLI — `--env` fonctionne, mais `APP_DEBUG` non

Contrairement à une intuition répandue, `php bin/console <commande> --env=synoptic` **fonctionne réellement** dans ce projet : le composant `symfony/runtime` (utilisé par `bin/console` via `autoload_runtime.php`) lit l'option `--env / -e` directement dans les arguments CLI et positionne `$_SERVER['APP_ENV']` **avant** Dotenv — ce n'est pas la vieille option `--env` de `symfony/console`, dépréciée et sans effet, avec laquelle on peut la confondre.

En revanche, **aucune règle `SetEnvIf` Apache ne s'applique en CLI** (pas de requête HTTP, pas de `Host`). Donc `APP_DEBUG` retombe systématiquement sur sa valeur par défaut (`true`, car `synoptic` n'est pas dans `prod_envs`) lors de toute commande lancée en SSH/cron, même avec `--env=synoptic` explicite.

Piège container — deux caches indépendants coexistent

Le nom de la classe container compilée par Symfony inclut le flag debug (`Kernel::getContainerClass()`, `vendor/symfony/http-kernel/Kernel.php`) :

```
$class = ... . ucfirst($this->environment) . ($this->debug ? 'Debug' : '') . 'Container';
```

Pour l'environnement `synoptic`, ça donne deux classes/caches **complètement indépendantes**, coexistant dans le même dossier `var/cache48a5079d/synoptic/` :

- `App_KernelSynopticContainer` (`debug=false` — utilisée par le vrai trafic HTTP en prod, grâce au `SetEnvIf APP_DEBUG=0`)
- `App_KernelSynopticDebugContainer` (`debug=true` — celle que construit n'importe quelle commande CLI lancée sans forcer `APP_DEBUG=0`)

Conséquence concrète : lancer `php bin/console cache:clear --env=synoptic` en SSH sur le serveur de prod affiche *"with debug true"* et ne vide/ne reconstruit que le cache **Debug**, jamais utilisé par le site réel. Les modifications ne semblent donc "pas prises en compte" après un `cache:clear` — en fait, c'est le mauvais cache qui a été vidé.

Commande correcte pour vider le cache réellement utilisé en prod :

```
APP_DEBUG=0 php bin/console cache:clear --env=synoptic
```

Comme pour `APP_ENV`, une vraie variable d'environnement shell est protégée contre toute réécriture par les fichiers `-.env.*`, donc ça force fiablement `debug=false` pour cette invocation.

Mise à jour 2026-07-20 (projet Synoptic uniquement) : ce contournement manuel n'est plus nécessaire sur Synoptic, remplacé par un correctif pérenne — voir section *"Suite (2026-07-20)"* plus bas. **BVP et Épione (Symfony 7) n'ont pas encore été adaptés à la date du 21/07/2026 et dépendent toujours de ce contournement manuel** : sur ces deux projets, la commande ci-dessus (avec le préfixe `APP_DEBUG=0`) reste la seule façon fiable de vider le cache réellement utilisé en prod.

Récapitulatif des fichiers modifiés (session du 2026-07-09)

Fichier	Changement
<code>config/packages/doctrine.yaml</code>	Ajout des blocs <code>when@synoptic:</code> et <code>when@preprod:</code> (dupliqués depuis <code>when@prod:</code>)
<code>.env.synoptic</code> (dev + prod)	Ligne <code>APP_ENV=prod</code> commentée
<code>.htaccess</code> (prod uniquement, non versionné)	Ajout de <code>SetEnvIf Host ^synoptic-v2.amshop.fr APP_DEBUG=0</code>
<code>clear-cache.sh</code>	Ajout de <code>APP_DEBUG=0</code> sur les invocations <code>--env=synoptic</code> / <code>--env=preprod</code>

Suite (2026-07-20, projet Synoptic) — correctif pérenne de `APP_DEBUG` , et profiler actif en prod

Remplacer le contournement CLI par un vrai correctif

Le contournement du 09/07 (préfixer `APP_DEBUG=0` à la main sur chaque commande CLI) fonctionne mais dépend de la mémoire de l'opérateur — un `cache:clear` lancé sans le préfixe reconstruit silencieusement le mauvais cache (`...DebugContainer`). Solution retenue : poser `APP_DEBUG=0` directement dans `.env.synoptic` (versionné, déployé sur le vrai serveur).

Ça ne contredit pas le mécanisme décrit plus haut, qui protège contre l'écrasement d'une vraie variable d'environnement déjà positionnée (`$_SERVER` , via `SetEnvIf` Apache ou un préfixe shell explicite) — c'est un mécanisme différent de la **cascade normale entre fichiers**

`.env.*` (`.env` → `.env.local` → `.env.$env` → `.env.$env.local`), où chaque fichier chargé plus tard écrase bien la valeur du précédent pour une même clé. Concrètement :

- **Trafic HTTP réel** : Apache pose déjà `APP_DEBUG=0` via `SetEnvIf` avant que Dotenv ne s'exécute → la nouvelle ligne dans `.env.synoptic` est inerte pour ce cas (même valeur, aucun conflit).
- **CLI (SSH/cron)** : aucune vraie variable d'environnement n'est positionnée en amont → c'est la valeur déclarée dans `.env.synoptic` (`0`) qui s'applique par défaut, sans plus jamais dépendre d'un préfixe manuel.
- **Poste de dev**, où `APP_ENV=synoptic` est aussi utilisé en local pour cibler ce tenant : `.env.synoptic.local` (non versionné) surcharge à nouveau avec `APP_DEBUG=1`, dernier fichier de la cascade à être chargé → le debug local reste actif, comme avant.

Vérifié après coup, sur le serveur de prod réel et sans aucun préfixe :

```
php bin/console debug:container --env=synoptic --parameter=kernel.debug
# => false
time php bin/console cache:clear --env=synoptic
# => "with debug false", ~3,4s
```

Le profiler tournait en prod, indépendamment du debug

Autre effet du même choix architectural (nom d'environnement = identifiant de tenant) :

`config/bundles.php` enregistrait `WebProfilerBundle` pour l'environnement `synoptic` explicitement (`'synoptic' => true`, en plus de `dev / test` — **ce n'est pas le comportement par défaut d'une nouvelle app Symfony**, qui ne l'active que pour `dev / test`). Et `config/packages/web_profiler.yaml` avait un bloc `when@synoptic:` activant `framework.profiler` **sans aucune condition liée au debug**.

Conséquence : le profiler écrivait un fichier par requête HTTP en vraie prod, avec du vrai trafic client, en continu, depuis des mois. Le `cache:clear` (dont l'étape finale supprime récursivement, fichier par fichier, l'ancien répertoire de cache renommé) mettait alors **plus de 15 minutes** à s'exécuter sur l'hébergement mutualisé — indice qui a mené au diagnostic : un répertoire `<hash>` abandonné depuis plusieurs semaines a été retrouvé sur le serveur, preuve qu'un précédent `cache:clear` n'avait jamais été mené à son terme.

Solution retenue, même principe que pour `APP_DEBUG` — gater la collecte du profiler sur un nouvel env var plutôt que sur le nom d'environnement :

```
# config/packages/web_profiler.yaml, bloc when@synoptic:
framework:
```

```
profiler:
  collect: '%env(bool:PROFILER_COLLECT)%'
```

`PROFILER_COLLECT=0` dans `.env.synoptic` (vraie prod), `PROFILER_COLLECT=1` dans `.env.synoptic.local` (poste de dev) — même schéma de surcharge que `APP_DEBUG` ci-dessus.

Récapitulatif des fichiers modifiés (session du 2026-07-20, commit 3b97a0b)

Fichier	Changement
<code>.env.synoptic</code> (dev + prod)	Ajout de <code>APP_DEBUG=0</code> et <code>PROFILER_COLLECT=0</code>
<code>.env.synoptic.local</code> (poste de dev, non versionné)	Ajout de <code>APP_DEBUG=1</code> et <code>PROFILER_COLLECT=1</code> (surcharges)
<code>config/packages/web_profiler.yaml</code>	<code>framework.profiler.collect</code> piloté par <code>%env(bool:PROFILER_COLLECT)%</code> sous <code>when@synoptic:</code>

État des autres projets utilisant ce même schéma (nom d'env = tenant)

Vérifié le 20/07/2026, ni l'un ni l'autre n'a encore reçu ce correctif — TODO posé dans le `CLAUDE.md` de chaque projet :

- **BVP** (`bvp/CLAUDE.md`) : a le piège `APP_DEBUG /CLI` (même cause, `APP_ENV=bvp` ≠ `prod`), mais **pas** le problème de profiler — `config/bundles.php` n'y active `WebProfilerBundle` que pour `dev / test` , conformément au comportement par défaut de Symfony (c'est bien Synoptic et Épione qui dérogent à ce défaut, pas l'inverse).
- **Épione Symfony 7.2** (`sf7.2_epione/CLAUDE.md`) : cumule les deux problèmes, `APP_DEBUG /CLI` et profiler actif en prod (même `when@epione:` sans condition de debug), profiler encore peu volumineux au 20/07/2026 (24 Ko) donc pas de crise immédiate contrairement à Synoptic, mais la faille de fond est identique.